CPSC-406 Report

Charlie Conner
Chapman University

April 12, 2026

Abstract

This is the place to write an abstract. The abstract should be a short summary of the report. It should be written in a way that makes it possible to understand the purpose of the report without reading it. You can write a dummy abstract first and replace it with a real one later in the semester.

Contents

1 Introduction **2**

2 Week by Week **3**
2.1 Week 0 (EXAMPLE) . . . 3
2.1.1 Notes . . . 3
2.1.2 Homework . . . 3
2.1.3 Exploration . . . 4
2.1.4 Questions . . . 4
2.2 Week 1 . . . 5
2.2.1 Notes . . . 5
2.2.2 Homework . . . 5
2.2.3 Exploration . . . 7
2.2.4 Questions . . . 8
2.3 Week 2 . . . 8
2.3.1 Notes . . . 8
2.3.2 Homework . . . 8
2.3.3 Exploration . . . 10
2.3.4 Questions . . . 10
2.4 Week 3 . . . 10
2.4.1 Notes . . . 10
2.4.2 Homework . . . 10
2.4.3 Exploration . . . 13
2.4.4 Questions . . . 13
2.5 Week 4 . . . 13
2.5.1 Notes . . . 13
2.5.2 Homework . . . 13
2.5.3 Exploration . . . 15
2.5.4 Questions . . . 15
2.6 Week 5 . . . 15
2.6.1 Notes . . . 15
2.6.2 Homework . . . 15
2.6.3 Exploration . . . 16

2.6.4 Questions . . . 17
2.7 Week 6 . . . 17
2.7.1 Notes . . . 17
2.7.2 Homework . . . 17
2.7.3 Exploration . . . 18
2.7.4 Questions . . . 19
2.8 Week 7 . . . 19
2.8.1 Notes . . . 19
2.8.2 Homework . . . 19
2.8.3 Exploration . . . 22
2.8.4 Questions . . . 22
2.9 Week 8 . . . 22
2.9.1 Notes . . . 22
2.9.2 Homework . . . 22
2.9.3 Exploration . . . 23
2.9.4 Questions . . . 24

3 Synthesis **24**

4 Evidence of Participation **25**

5 Conclusion **25**

1 Introduction

This report will document your learning throughout the course. It will be a collection of your notes, homework solutions, and critical reflections on the content of the course. Something in between a semester-long take home exam and your own lecture notes.[1]

To write your own report, you start from `report.tex` which is available in the course repo. For guidance on how to do this read on and also consult `latex-example.tex` which is also available in the repo. Also check out the usual resources (Google, Stackoverflow, LLM, etc). It was never as easy as now to learn a new programming lanugage (which, btw, LaTeX is).

For writing LaTeX with VSCode, consider using the LaTeX Workshop extension.

There will be deadlines during the semester, graded mostly for completeness. That means that you will get the points if you submit in time and are on the right track, independently of whether the solutions are technically correct. You will have the opportunity to revise your work for the final submission of the full report.

The full report is due at the end of the finals week. It will be graded according to the following guidelines.

Grading guidelines (see also below):

- Is typesetting and layout professional?
- Is the technical content, in particular the homework, correct?
- Did the student find interesting references [BLA] and cites them throughout the report?

[1] One purpose of giving the report the form of lecture notes is that self-explanation is a technique proven to help with learning, see Chapter 6 of Craig Barton, How I Wish I'd Taught Maths, and references therein. In fact, the report can lead you from self-explanation (which is what you do for the weekly deadline) to explaining to others (which is what you do for the final submission). Another purpose is to help those of you who want to go on to graduate school to develop some basic writing skills. A report that you could proudly add to your application to graduate school (or a job application in industry) would give you full points.

- Do the notes reflect understanding and critical thinking?
- Does the report contain material related to but going beyond what we do in class?
- Are the questions interesting?

Do not change the template (fontsize, width of margin, spacing of lines, etc) without asking your first.

2 Week by Week

2.1 Week 0 (EXAMPLE)

Week 1 aligns with the first week of the semester.

If you think that the writing flows better if you merge the sections "Notes" and "Homework", you can do so, but keep the heading for "Comments and Questions".

2.1.1 Notes

This section is optional.

Our experience is that writing notes is a great way to learn. You can use this section to write your own notes and showcase your own understanding.

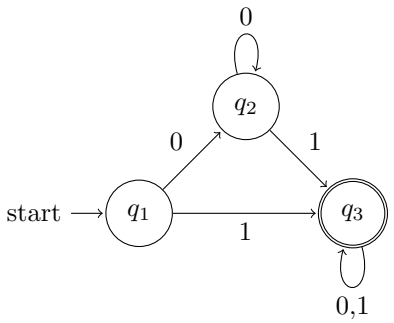
2.1.2 Homework

This section will typically contain Homework problems. You should write up your solutions in LaTeX. You can use the `lstlisting` environment to include code. You can use Excalidraw for drawings. Pictures from handwritten drawings are acceptable if the drawings are of high quality (pictures from rough notes and quick sketches are likely to loose you points).

Make sure that this section can be read without referring back to the homework question. Introduce the question/problem and repeat it in your own words. Make sure to typset your homework in a way that makes it clear what the question and what the answer is. Present it as a worked example would be presented in a textbook.

Also explain what you learn from the homework. Each homework was carefully drafted to bring home a particular teaching point. Make sure to explain what this point is. Relate it to the big questions mentioned above.

In case you want to draw automata in LaTeX, you can use the tikz package. Here is an example of a simple automaton:



By the way, ChatGPT is quite good at outputting tikz code. Another alternative package is xymatrix.

2.1.3 Exploration

Here are some hints. Why is this material included in the course? What are the big questions that motivate the study of this subject? How does this material connect to broader themes or issues in the field? What practical or theoretical problems can be addressed through an understanding of these topics? A great way to test whether you understand the material is to make your own exercises and answer them. Material related to but going beyond what we do in class is welcome.

Feel free to use your favourite LLM to help you build a mental landscape of the subject. Think of LLMs as an extension of Wikipedia and Google, a tool you should be using as introduction to any subject. If you didn't check with Google, Wikipedia and GPT, you are not ready to write your own notes. On the other hand, while using these resources is necessary, you need to always exercise your own critical thinking and you are always responsible for what you write.

2.1.4 Questions

Ask at least one **interesting question**[2] on the lecture notes. Also post the question on the Discord channel so that everybody can see and discuss the questions.

[2]It is important to learn to ask *interesting* questions. There is no precise way of defining what is meant by interesting. You can only learn this by doing. An interesting question comes typically in two parts. Part 1 (one or two sentences) sets the scene. Part 2 (one or two sentences) asks the question. A good question strikes the right balance between being specific and technical on the one hand and open ended on the other hand. A question that can be answered with yes/no is not an interesing question.

2.2 Week 1

2.2.1 Notes

This section is optional.

2.2.2 Homework

Exercise 1 (Word processing with DFAs). Given two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ over $\Sigma = \{a, b\}$, determine which words are accepted/refused and describe the accepted languages.

DFA $\mathcal{A}_1$. States $\{1, 2, 3, 4\}$, start state 1, accepting state 3:

	a	b
1	2	4
2	2	3
3	2	2
4	4	4

DFA $\mathcal{A}_2$. States $\{1, 2, 3\}$, start state 1, accepting state 3:

	a	b
1	2	1
2	3	1
3	3	1

Part 1: Tracing words. For each word, I walked through the transition table one character at a time and recorded the states:

w	$\mathcal{A}_1$ trace	$\mathcal{A}_1$?	$\mathcal{A}_2$ trace	$\mathcal{A}_2$?
aaa	$1 \to 2 \to 2 \to 2$	No	$1 \to 2 \to 3 \to 3$	Yes
aab	$1 \to 2 \to 2 \to 3$	Yes	$1 \to 2 \to 3 \to 1$	No
aba	$1 \to 2 \to 3 \to 2$	No	$1 \to 2 \to 1 \to 2$	No
abb	$1 \to 2 \to 3 \to 2$	No	$1 \to 2 \to 1 \to 1$	No
baa	$1 \to 4 \to 4 \to 4$	No	$1 \to 1 \to 2 \to 3$	Yes
bab	$1 \to 4 \to 4 \to 4$	No	$1 \to 1 \to 2 \to 1$	No
bba	$1 \to 4 \to 4 \to 4$	No	$1 \to 1 \to 1 \to 2$	No
bbb	$1 \to 4 \to 4 \to 4$	No	$1 \to 1 \to 1 \to 1$	No

Part 2: Describing the languages. **$L(\mathcal{A}_1)$: Words that start with a and end with an odd-length block of consecutive b's.**

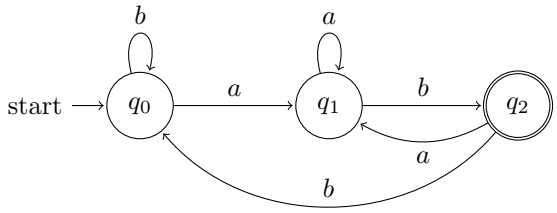
Starting with b traps us in state 4, so the word must start with a. After that, reading a resets us to state 2, and reading b flips between states 2 and 3. So we accept when the number of b's since the last a is odd.

$L(\mathcal{A}_2)$: Words that end with aa.

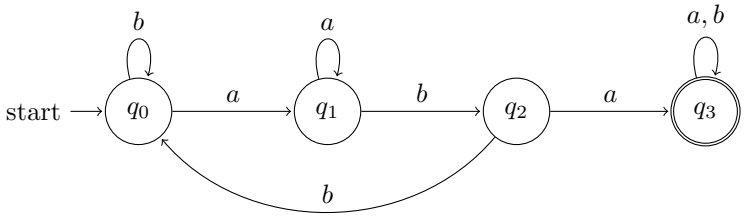
Any b resets us to state 1, and a's move us along $1 \to 2 \to 3$ (staying in 3 on more a's). So we accept when the word ends with two or more a's.

Exercise 2 (Designing DFAs). Design DFAs over $\Sigma = \{a, b\}$ for each of the following languages.

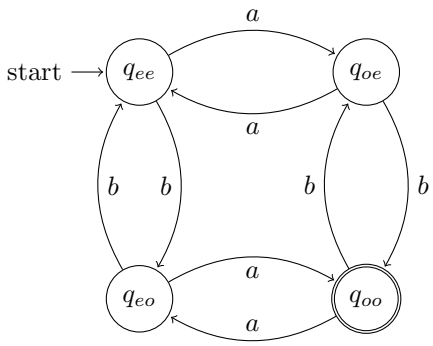
(a) All words ending with ab. Three states: q_0 (no progress), q_1 (last char was a), q_2 (saw ab, accepting).



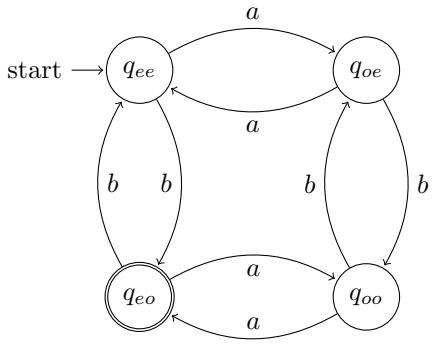
(b) All words containing aba. Track how much of aba we've matched. Once found, we stay accepting (trap state).



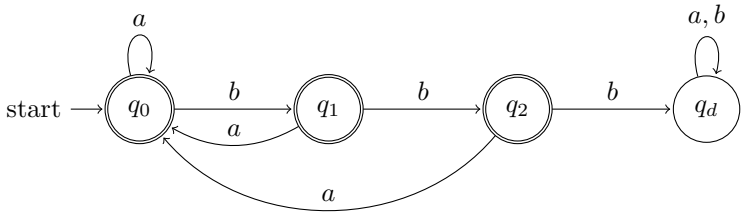
(c) All words with an odd number of a's and an odd number of b's. Four states for each combination of (parity of a's, parity of b's). Reading a flips the a-parity, reading b flips the b-parity.



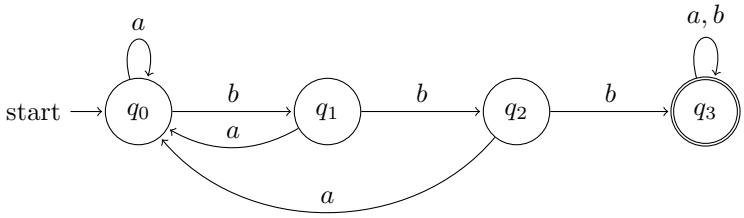
(d) All words with an even number of a's and an odd number of b's. Same DFA as (c), just with q_{eo} as the accepting state instead of q_{oo}.



(e) All words where any three consecutive characters contain at least one a. This is the same as saying the word never contains bbb. We count consecutive b's.



(f) All words containing bbb. Same idea as (e) but we count consecutive b's until we hit three, then stay accepting.



Comparison.

- **(e) and (f) are complements.** Same states and transitions, just with accept/reject swapped. To get the complement of a regular language, flip the accepting states.
- **(c) and (d) share the same structure.** Same transitions, different accepting state. One automaton skeleton, two different languages.
- **(a) vs. (b).** For end-patterns (a), the DFA cycles back when the match breaks. For substrings (b), once we find it we're done and sit in the accepting state.

2.2.3 Exploration

I did research on DFAs to understand them better, here are some interesting things I found.

DFAs can only remember which state they're in. No extra memory, no stack, nothing. That sounds limiting, but it's actually why they're useful.

Where DFAs show up. Regex engines (`grep`, Python `re`, etc.) compile patterns into automata. Compilers use DFAs to break source code into tokens. Even traffic lights and vending machines are basically finite state machines.

Why the limitation is good. Since DFAs have finite memory, they always finish running, you can always check if two DFAs accept the same language, and you can always shrink a DFA to its smallest form. These are things you lose with stronger models. With a Turing machine, for example, you can't even tell if it will stop running.

DFAs and pure functions. A pure function always gives the same output for the same input, with no hidden state or side effects. That makes it easy to test and reason about. DFAs work the same way: $\delta(q, a)$ just looks at the current state and symbol, nothing else. You can check every possible behavior because there are only finitely many states. You give up some power but everything is easy to follow.

2.2.4 Questions

We showed in Exercise 2 that different-looking DFAs can accept the same language (e.g., the complement pair for *bbb*). Since every regular language has a unique minimal DFA, can you always prove two DFAs are equivalent just by minimizing both and comparing?

2.3 Week 2

2.3.1 Notes

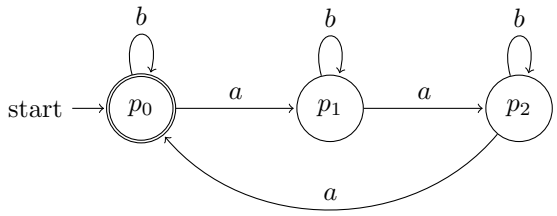
This section is optional.

2.3.2 Homework

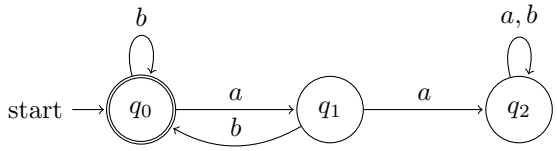
Exercise 2

Consider two DFAs $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ over $\Sigma = \{a, b\}$. We construct their intersection automaton using the product construction.

DFA $\mathcal{B}^{(1)}$. States $\{p_0, p_1, p_2\}$, start state p_0, accepting state p_0:



DFA $\mathcal{B}^{(2)}$. States $\{q_0, q_1, q_2\}$, start state q_0, accepting state q_0:



Part 1: Describing the languages. $L(\mathcal{B}^{(1)})$: Words where the number of a's is divisible by 3.

Since b self-loops at every state, only a's matter. Reading a cycles $p_0 \to p_1 \to p_2 \to p_0$, and p_0 is the only accepting state, so we accept after $0, 3, 6, \ldots$ letters a:

$$L(\mathcal{B}^{(1)}) = \{\, w \in \{a, b\}^* : \#_a(w) \equiv 0 \pmod 3 \,\}.$$

$L(\mathcal{B}^{(2)})$: Words where every a is immediately followed by b.

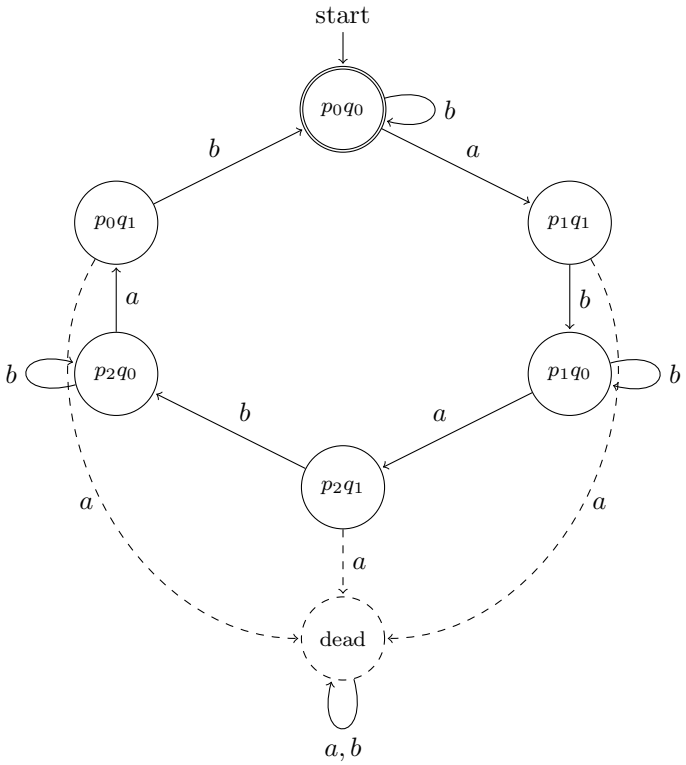
Reading a from q_0 goes to q_1, which "waits" for a b. Getting b returns to q_0, but another a traps us in the dead state q_2. So every a must have a b right after it, meaning valid words are sequences of ab and b blocks:

$$L(\mathcal{B}^{(2)}) = (ab + b)^*.$$

Part 2: Intersection automaton $\mathcal{B}$. Using the product construction, I built $\mathcal{B} = (Q, \Sigma, \delta, (p_0, q_0), F)$ where $Q = \{p_0, p_1, p_2\} \times \{q_0, q_1, q_2\}$ (9 states), $\delta((p_i, q_j), x) = (\delta_1(p_i, x),\ \delta_2(q_j, x))$, and $F = \{(p_0, q_0)\}$ since we need both components accepting. Here is the transition table:

	a	b
$(p_0, q_0)^\star$	(p_1, q_1)	(p_0, q_0)
(p_0, q_1)	(p_1, q_2)	(p_0, q_0)
(p_0, q_2)	(p_1, q_2)	(p_0, q_2)
(p_1, q_0)	(p_2, q_1)	(p_1, q_0)
(p_1, q_1)	(p_2, q_2)	(p_1, q_0)
(p_1, q_2)	(p_2, q_2)	(p_1, q_2)
(p_2, q_0)	(p_0, q_1)	(p_2, q_0)
(p_2, q_1)	(p_0, q_2)	(p_2, q_0)
(p_2, q_2)	(p_0, q_2)	(p_2, q_2)

All nine states are reachable. The six states with q_0 or q_1 form a hexagonal cycle alternating a, b, a, b, a, b. The three q_2 states are a dead zone since none of them can reach (p_0, q_0). I drew the dead zone as one dashed node below; see the table for full detail.



So we need both conditions: every a followed by b, and the number of a's divisible by 3. Since each a lives in an ab-block, the number of ab-blocks must be a multiple of 3:

$$L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)}) = \{\, w \in (ab + b)^* : \text{the number of } ab\text{-blocks is divisible by } 3 \,\}.$$

Part 3: $L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)})$. We want to show $\hat{\delta}((p_0, q_0), w) = (\hat{\delta}_1(p_0, w),\ \hat{\delta}_2(q_0, w))$ for any word w. By induction on $|w|$:

Base case. $w = \varepsilon$: $\hat{\delta}((p_0, q_0), \varepsilon) = (p_0, q_0) = (\hat{\delta}_1(p_0, \varepsilon),\ \hat{\delta}_2(q_0, \varepsilon))$. ✓

Inductive step. Assume it holds for w. For any $a \in \Sigma$:

$$\begin{aligned}
\hat{\delta}((p_0, q_0), wa) &= \delta\big(\hat{\delta}((p_0, q_0), w),\, a\big) \\
&= \delta\big((\hat{\delta}_1(p_0, w),\, \hat{\delta}_2(q_0, w)),\, a\big) && \text{(ind. hyp.)} \\
&= \big(\delta_1(\hat{\delta}_1(p_0, w), a),\; \delta_2(\hat{\delta}_2(q_0, w), a)\big) && \text{(def. of } \delta\text{)} \\
&= \big(\hat{\delta}_1(p_0, wa),\; \hat{\delta}_2(q_0, wa)\big).
\end{aligned}$$

Then $w \in L(\mathcal{B})$ iff we land in $F_1 \times F_2$, iff both components accept, iff $w \in L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)})$. □

Part 4: $L(\mathcal{B}') = L(\mathcal{B}^{(1)}) \cup \overline{L(\mathcal{B}^{(2)})}$. We reuse the same product automaton from Part 2 and just change which states are accepting. Complementing $\mathcal{B}^{(2)}$ flips its accept states: $\overline{F_2} = \{q_1, q_2\}$. For the union we accept when the first component accepts *or* the second rejects:

$$F' = \{(p_i, q_j) : p_i \in F_1 \text{ or } q_j \notin F_2\} = \{(p_i, q_j) : p_i = p_0 \text{ or } q_j \in \{q_1, q_2\}\}.$$

That gives $F' = \{(p_0, q_0), (p_0, q_1), (p_0, q_2), (p_1, q_1), (p_1, q_2), (p_2, q_1), (p_2, q_2)\}$. Only (p_1, q_0) and (p_2, q_0) are non-accepting. By the same induction as Part 3, $w \in L(\mathcal{B}')$ iff $\hat{\delta}_1(p_0, w) \in F_1$ or $\hat{\delta}_2(q_0, w) \notin F_2$, which is $L(\mathcal{B}^{(1)}) \cup \overline{L(\mathcal{B}^{(2)})}$. □

2.3.3 Exploration

After some research I noticed that regex tools like `grep` and Python `re` have union (`|`) but no intersection or complement operator. The theory says these operations keep you inside regular languages, but the product construction squares the number of states, so most engines just skip it.

2.3.4 Questions

The complement of a DFA is easy (just flip accepting states). Why isn't there an equally simple trick for intersection without building the full product?

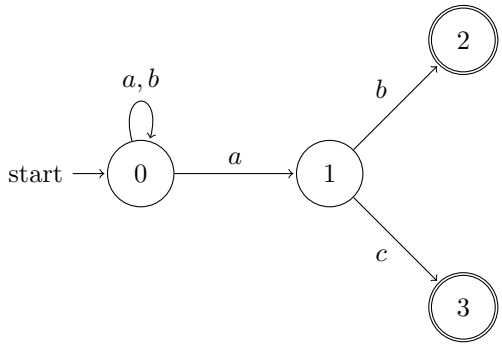
2.4 Week 3

2.4.1 Notes

This section is optional.

2.4.2 Homework

Homework 1. For the alphabet $\Sigma = \{a, b, c\}$, consider the following NFA $\mathcal{A}$:



Part 1: Regular expression. State 0 loops on a and b, so we can read any string of a's and b's while staying in 0. Then reading a nondeterministically moves to 1, and from 1 we read b to reach accepting state 2 or c to reach accepting state 3. There are no transitions out of 2 or 3, and no c-transition from 0. So the language is all words over $\{a, b, c\}$ that end with ab or ac:

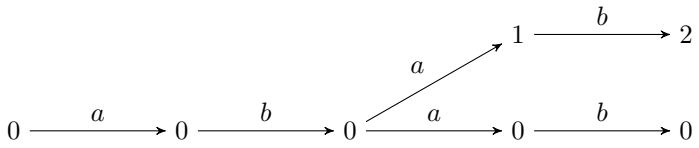
$$L(\mathcal{A}) = (a + b)^* \cdot a \cdot (b + c).$$

Part 2: NFA specification. $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ where:

- $Q = \{0, 1, 2, 3\}$
- $\Sigma = \{a, b, c\}$
- $q_0 = 0$
- $F = \{2, 3\}$
- The transition function $\delta \colon Q \times \Sigma \to \mathcal{P}(Q)$ is:

	a	b	c
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\{3\}$
2	$\emptyset$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

Part 3: All paths for $v = abab$. We trace all paths that process the full word $v = abab$. Paths that get stuck early are omitted.



There are two complete paths:

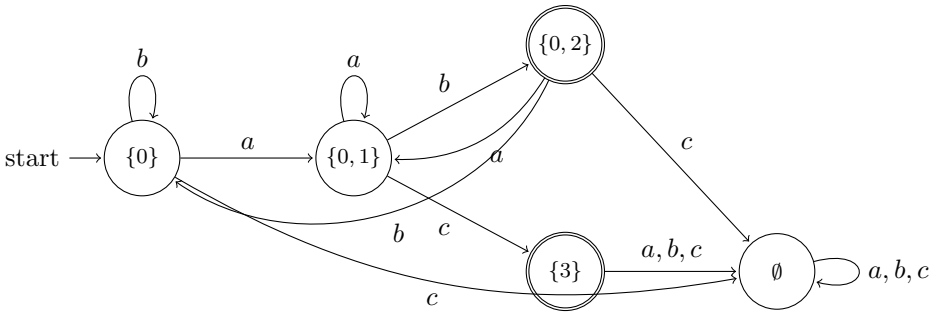
1. $0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 0 \xrightarrow{b} 0$ (not accepting)
2. $0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 1 \xrightarrow{b} 2$ (accepting)

Since at least one path ends in an accepting state, $v = abab$ is accepted by $\mathcal{A}$.

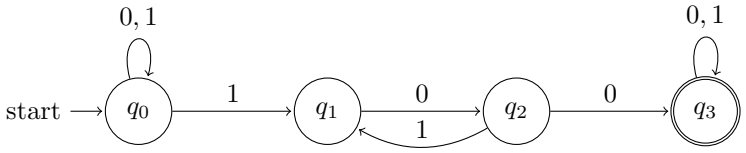
Part 4: Power set construction. Starting from $\{0\}$ and building reachable states:

	a	b	c
$\to \{0\}$	$\{0, 1\}$	$\{0\}$	$\emptyset$
$\{0, 1\}$	$\{0, 1\}$	$\{0, 2\}$	$\{3\}$
$*\{0, 2\}$	$\{0, 1\}$	$\{0\}$	$\emptyset$
$*\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

States containing 2 or 3 are accepting: $\{0, 2\}$ and $\{3\}$. The trap state $\emptyset$ appears because $\delta(0, c) = \emptyset$. The DFA $\mathcal{A}^{\mathrm{D}}$:



Homework 2. For the alphabet $\Sigma = \{0, 1\}$, consider the following NFA $\mathcal{A}$:



The NFA has $Q = \{q_0, q_1, q_2, q_3\}$, start state q_0, accepting state q_3, and transitions:

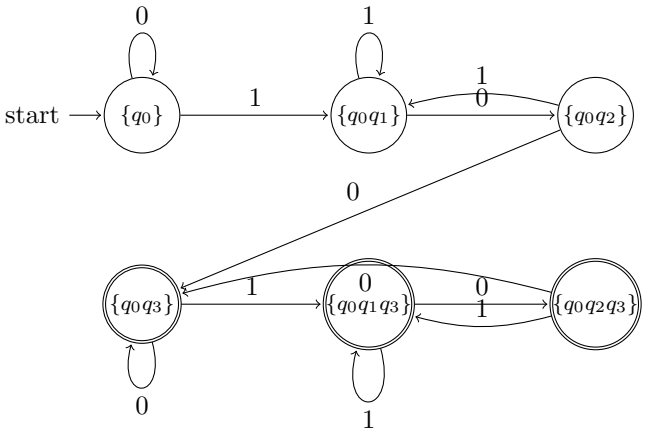
	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\{q_1\}$
q_3	$\{q_3\}$	$\{q_3\}$

The NFA nondeterministically guesses where the substring 100 starts. From q_0 it reads 1 to enter q_1, then 0 to q_2, then 0 to accepting q_3. The $q_2 \to q_1$ transition on 1 lets it retry, but q_0's self-loop already handles restarting. So $L(\mathcal{A})$ is the set of all words over $\{0, 1\}$ containing 100 as a substring.

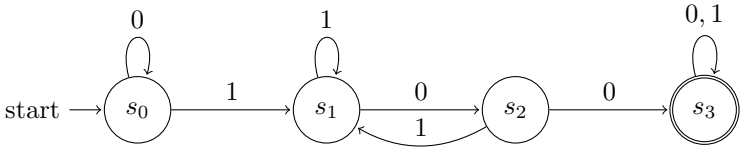
Part 1: Power set construction. Building reachable states from $\{q_0\}$:

	0	1
$\to \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1\}$
$*\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
$*\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
$*\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$

States containing q_3 are accepting. The DFA $\mathcal{A}^{\mathrm{D}}$ has 6 reachable states:



Part 2: A smaller DFA. Yes. Since the language is "contains 100," we just need to track how much of the pattern 100 we have matched so far. This gives a 4-state DFA:



Here s_0 means no progress, s_1 means the last character was 1, s_2 means the last two were 10, and s_3 means we found 100. This DFA has 4 states compared to the 6 from the power set construction. The two extra states in $\mathcal{A}^{\mathrm{D}}$ come from tracking NFA-specific information (which subset of $\{q_0, q_1, q_2, q_3\}$ we are in) that is not needed once we realize the language is just "contains 100."

2.4.3 Exploration

The power set construction is pretty straightforward, you just follow the steps and you get a DFA. But it doesn't always give you the smallest one. In Homework 2 I got 6 states, but a DFA I built by hand only needed 4.

NFAs also seem way easier to design than DFAs. You just think about one pattern at a time and let the nondeterminism figure out the rest. Then you can always convert to a DFA afterward if you need to.

2.4.4 Questions

The power set construction gave us 5 states for Homework 1 and 6 for Homework 2, but the NFAs only had 4 states each. When does the DFA end up smaller than the NFA, and when does it end up bigger?

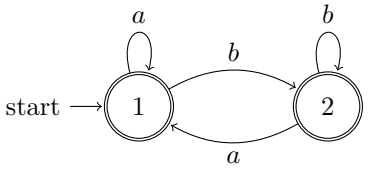
2.5 Week 4

2.5.1 Notes

This section is optional.

2.5.2 Homework

Exercise 1 (Kleene's Algorithm). Given a DFA with states 1 (start) and 2, both accepting, over $\Sigma = \{a, b\}$:



Applying Kleene's algorithm with $n = 2$ states using $R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)}(R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}$.

Base case ($k = 0$).

$$R_{11}^{(0)} = \varepsilon + a, \qquad R_{12}^{(0)} = b,$$
$$R_{21}^{(0)} = a, \qquad R_{22}^{(0)} = \varepsilon + b.$$

Induction $k = 1$ (through state 1).

$$R_{11}^{(1)} = (\varepsilon + a) + (\varepsilon + a)(\varepsilon + a)^*(\varepsilon + a) = a^*,$$
$$R_{12}^{(1)} = b + (\varepsilon + a)(\varepsilon + a)^* b = a^* b,$$
$$R_{21}^{(1)} = a + a(\varepsilon + a)^*(\varepsilon + a) = aa^*,$$
$$R_{22}^{(1)} = (\varepsilon + b) + a(\varepsilon + a)^* b = \varepsilon + a^* b.$$

Induction $k = 2$ (through states 1 and 2). Using $(\varepsilon + a^*b)^* = (a^*b)^*$:

$$R_{11}^{(2)} = a^* + a^* b \cdot (a^* b)^* \cdot aa^*,$$
$$R_{12}^{(2)} = a^* b + a^* b \cdot (a^* b)^* \cdot (\varepsilon + a^* b) = (a^* b)^+.$$

Final result. Since $F = \{1, 2\}$: $R_{\mathcal{A}} = R_{11}^{(2)} + R_{12}^{(2)} = a^* + a^* b(a^* b)^* aa^* + (a^* b)^+$. These three terms cover strings of only a's, strings with b's ending in a, and strings ending in b, so $R_{\mathcal{A}} = (a + b)^*$. This makes sense since both states are accepting and the DFA is complete.

Exercise 2 (ITALC 4.4.2: DFA Minimization). DFA with states $\{A, \ldots, I\}$, start A, accepting $\{C, F, I\}$, over $\Sigma = \{0, 1\}$:

	0	1
$\to A$	B	E
B	C	F
$^*C$	D	H
D	E	H
E	F	I
$^*F$	G	B
G	H	B
H	I	C
$^*I$	A	E

Part (a): Table of distinguishabilities. *Basis.* Mark all pairs where exactly one state is accepting ($6 \times 3 = 18$ marked pairs).

Round 1. Nine more pairs get marked because a successor on 0 or 1 hits an already-marked pair: (A, B), (A, E), (A, H), (B, D), (B, G), (D, E), (D, H), (E, G), (G, H).

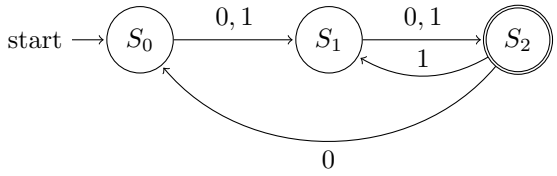
Round 2. The remaining 9 pairs all have successors within the unmarked set:

Pair	on 0	on 1
(A, D)	(B, E)	(E, H)
(A, G)	(B, H)	(B, E)
(B, E)	(C, F)	(F, I)
(B, H)	(C, I)	(C, F)
(D, G)	(E, H)	(B, H)
(E, H)	(F, I)	(C, I)
(C, F)	(D, G)	(B, H)
(C, I)	(A, D)	(E, H)
(F, I)	(A, G)	(B, E)

Fixed point. Equivalence classes: $\{A, D, G\}$, $\{B, E, H\}$, $\{C, F, I\}$.

Part (b): Minimum-state DFA. Let $S_0 = \{A, D, G\}$, $S_1 = \{B, E, H\}$, $S_2 = \{C, F, I\}$. Start S_0, accepting S_2.

	0	1
$\to S_0$	S_1	S_1
S_1	S_2	S_2
$^*S_2$	S_0	S_1



Reduces from 9 states to 3. S_0 resets, S_1 is one step from accepting, S_2 accepts. From S_2, input 0 cycles back to S_0 while 1 goes to S_1.

2.5.3 Exploration

Kleene's algorithm gave a messy regex for a 2-state DFA that just accepts everything. At the same time, minimization took 9 states down to 3. One method tries to shrink the automaton, the other converts it to a regex that can blow up exponentially. Neither representation is always smaller, it depends on the language.

2.5.4 Questions

Is there a way to make Kleene's algorithm skip work when the answer is obviously simple, like when every state accepts?

2.6 Week 5

2.6.1 Notes

This section is optional.

2.6.2 Homework

Exercise 5 (Pumping lemma). Use the pumping lemma to prove that the following four languages are not regular.

Language $L_1 = \{a^n b^m \mid n > m \geq 0\}$**.** *Assumption.* Suppose L_1 is regular with pumping length p.

Pumping. Choose $w = a^{p+1}b^p$. Since $p + 1 > p$, we have $w \in L_1$ and $|w| = 2p + 1 \geq p$. By condition 2, $|xy| \leq p$, so xy lies entirely within the leading a's. Write $y = a^t$ with $t \geq 1$. Pump down with $k = 0$:

$$xz = a^{p+1-t}\, b^p.$$

Since $t \geq 1$, the number of a's satisfies $p + 1 - t \leq p$, so $xz \notin L_1$.

Contradiction. The pumping lemma requires $xy^k z \in L_1$ for all $k \geq 0$, but $xz \notin L_1$. Therefore L_1 is not regular. □

Language $L_2 = \{a^{n^2} \mid n \geq 1\}$**.** *Assumption.* Suppose L_2 is regular with pumping length p.

Pumping. Choose $w = a^{p^2}$. Then $w \in L_2$ and $|w| = p^2 \geq p$. By condition 2, $|y| \leq p$. Let $|y| = t$ with $1 \leq t \leq p$. Pump up with $k = 2$:

$$|xy^2z| = p^2 + t.$$

Since $1 \leq t \leq p$, we have $p^2 < p^2 + t \leq p^2 + p < p^2 + 2p + 1 = (p + 1)^2$. So $p^2 + t$ is strictly between consecutive perfect squares and is not itself a perfect square, meaning $xy^2z \notin L_2$.

Contradiction. The pumping lemma is violated, so L_2 is not regular. □

Language $L_3 = \{a^p \mid p$ **is prime**$\}$**.** *Assumption.* Suppose L_3 is regular with pumping length n.

Pumping. Pick a prime $q \geq n+2$ and set $w = a^q$. Then $w \in L_3$ and $|w| = q \geq n$. Let $|y| = t$ with $1 \leq t \leq n$. Pump with $k = q + 1$:

$$|xy^{q+1}z| = q + qt = q(1 + t).$$

Since $t \geq 1$ we have $1 + t \geq 2$, and since $q \geq 3$, both $q \geq 3$ and $1 + t \geq 2$, so $q(1 + t)$ is composite. So $xy^{q+1}z \notin L_3$.

Contradiction. The pumping lemma is violated, so L_3 is not regular. □

Language $L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}$**.** *Assumption.* Suppose L_4 is regular with pumping length n.

Pumping. Choose $w = a^n b^n a^{2n}$. With $k = m = n$ we have $k + m = 2n$, so $w \in L_4$ and $|w| = 4n \geq n$. By condition 2, $|xy| \leq n$, so xy lies entirely in the first a^n block. Write $y = a^t$ with $t \geq 1$. Pump up with $k = 2$:

$$xy^2z = a^{n+t}\, b^n\, a^{2n}.$$

If $xy^2z \in L_4$ then the trailing a-block has to equal the leading a-block plus the b-block, so $2n = (n+t)+n = 2n + t$, which gives $t = 0$. But $t \geq 1$.

Contradiction. The pumping lemma is violated, so L_4 is not regular. □

2.6.3 Exploration

All four languages fail the pumping lemma because a DFA can't count. Finitely many states means it eventually loops and forgets how far in it is.

The fix is adding a stack. For $a^n b^n$, push on every a, pop on every b, accept if the stack empties. That's a pushdown automaton. L_1 and L_4 work the same way.

L_2 and L_3 are different. Checking perfect squares or primes takes more than a stack. Those need a Turing machine.

2.6.4 Questions

The pumping lemma can prove a language isn't regular, but it can't confirm one is regular. Is there a tool that works in both directions?

2.7 Week 6

2.7.1 Notes

This section is optional.

2.7.2 Homework

Problem A (Turing Machines). Design Turing machines for the following three tasks over $\Sigma = \{0, 1\}$.

A1: Accept $L = \{10^n : n \in \mathbb{N}\}$ and return 10^{n+1}. The machine verifies the input starts with 1 followed by only 0's, then appends an extra 0 at the end.

States: q_0 (start), q_1 (scanning 0's), q_3 (accept). $F = \{q_3\}$.

δ	0	1	B
q_0	—	$(q_1, 1, R)$	—
q_1	$(q_1, 0, R)$	—	$(q_3, 0, R)$
$^*q_3$	—	—	—

Trace on input 100 *(i.e.* 10^2*):*

$$q_0100 \vdash 1q_100 \vdash 10q_10 \vdash 100q_1B \vdash 1000q_3B.$$

Output: $1000 = 10^3$. The machine rejects anything not of the form 10^n because q_0 requires a 1 first and q_1 only allows 0's.

A2: Accept $L = \{10^n : n \in \mathbb{N}\}$ and return 1. The machine verifies the input is of the form 10^n, then moves back left and erases all the 0's by overwriting them with blanks.

States: q_0 (start), q_1 (scan right through 0's), q_2 (move left erasing 0's), q_3 (accept). $F = \{q_3\}$.

δ	0	1	B
q_0	—	$(q_1, 1, R)$	—
q_1	$(q_1, 0, R)$	—	(q_2, B, L)
q_2	(q_2, B, L)	$(q_3, 1, R)$	—
$^*q_3$	—	—	—

Trace on input 100*:*

$$q_0100 \vdash 1q_100 \vdash 10q_10 \vdash 100q_1B \vdash 10q_20 \vdash 1q_2B \vdash q_21 \vdash 1q_3B.$$

When q_2 reads 0 it writes B and moves left; when it reaches the 1 it transitions to the accepting state q_3. Output: 1.

A3: Accept all binary strings and swap 0's and 1's. The machine scans left to right, replacing every 0 with 1 and every 1 with 0, halting when it hits the first blank.

States: q_0 (start/scan, also accepting). $F = \{q_0\}$.

δ	0	1	B
$^*q_0$	$(q_0, 1, R)$	$(q_0, 0, R)$	—

Trace on input 1010*:*

$$q_0 1010 \vdash 0 q_0 010 \vdash 01 q_0 10 \vdash 010 q_0 0 \vdash 0101 q_0 B.$$

The machine halts in q_0 (accepting). Output: 0101. This TM accepts every binary string since q_0 is the sole state and is accepting; it halts whenever it encounters B.

Exercise 1 (Decidability). For each statement, determine whether it is true or false. If true, give an argument; if false, give a counterexample.

1. If L_1 and L_2 are decidable, then so is $L_1 \cup L_2$. *True.* Since L_1 and L_2 are decidable, there exist TMs M_1 and M_2 that always halt and decide them. Construct a TM M for $L_1 \cup L_2$: on input w, run M_1 on w. If M_1 accepts, accept. If M_1 rejects, run M_2 on w. If M_2 accepts, accept; otherwise reject. M always halts because both M_1 and M_2 always halt, and M accepts w iff $w \in L_1$ or $w \in L_2$. □

2. If L is decidable, then so is $\overline{L}$. *True.* Let M be a TM that decides L (always halts). Build M' that runs M on input w and flips the answer: if M accepts, M' rejects; if M rejects, M' accepts. M' always halts and accepts exactly the strings not in L. □

3. If L is decidable, then so is L^*. *True.* Since L is decidable, we have a TM M that decides L. Construct M' for L^*: on input w of length n, systematically try every way to partition w into substrings $w_1 w_2 \cdots w_k$ (there are finitely many partitions, at most 2^{n-1} for a string of length n). For each partition, run M on every piece w_i. If there exists a partition where M accepts all pieces, accept. If no partition works, reject. Since M always halts on every piece and there are finitely many partitions, M' always halts. (Also, $\varepsilon \in L^*$ by definition, so M' accepts ε directly.) □

4. If L_1 and L_2 are r.e., then $L_1 \cup L_2$ is r.e. *True.* Let M_1 recognize L_1 and M_2 recognize L_2. Build M: on input w, simulate M_1 and M_2 in parallel (dovetailing, alternating one step of M_1 with one step of M_2). If either machine ever accepts, M accepts. If $w \in L_1 \cup L_2$, at least one of M_1 or M_2 will eventually accept, so M accepts. If $w \notin L_1 \cup L_2$, M may run forever, which is fine for r.e. □

5. If L is r.e., then so is $\overline{L}$. *False.* Counterexample: let $L = \{\langle M, w\rangle : M \text{ halts on input } w\}$ (the halting language). L is r.e., just simulate M on w, and accept if M halts. However, $\overline{L}$ is not r.e. If both L and $\overline{L}$ were r.e., we could run recognizers for L and $\overline{L}$ in parallel; one must eventually accept, so L would be decidable. But the halting problem is undecidable, contradiction. □

6. If L is r.e., then so is L^*. *True.* Let M be a TM that recognizes L. Build M': on input w, nondeterministically guess a partition of w into substrings $w_1 w_2 \cdots w_k$, then simulate M on each w_i in parallel (dovetailing). If M accepts all pieces, accept. If $w \in L^*$, there exists a partition where every piece is in L, and since M recognizes L, it will eventually accept each piece, so M' eventually accepts. Since nondeterministic TMs recognize exactly the r.e. languages, L^* is r.e. □

2.7.3 Exploration

Turing machines are the natural next step after DFAs. The key difference is the read/write tape: a DFA can only move right and never change anything, but a TM can go back, erase, and write new symbols. That's enough to go from recognizing patterns to actually computing things.

The decidability exercise shows that closure properties depend on how powerful your machines are. Decidable languages are closed under complement (just flip accept/reject), but r.e. languages are not. Flipping only works when the machine always halts. If a TM might loop forever on strings outside L, there's no answer to swap.

2.7.4 Questions

Is every non-r.e. language just the complement of some r.e. language, or are there languages that fall outside both classes entirely?

2.8 Week 7

2.8.1 Notes

This section is optional.

2.8.2 Homework

Exercise 2 (Classifying languages by decidability). For each of the following languages, determine whether it is decidable, recursively enumerable (r.e.), or has recursively enumerable complement (co-r.e.).

1. $L_1 = \{\langle M \rangle : M \textbf{ halts on input } \langle M \rangle\}$. L_1 is r.e. but not decidable and not co-r.e.

R.e.: Construct a TM R that, on input $\langle M \rangle$, simulates M on $\langle M \rangle$. If the simulation halts, accept. If M does not halt on itself, R runs forever. This is a recognizer for L_1, so L_1 is r.e.

Not decidable: Suppose for contradiction that some decider D decides L_1: on input $\langle M \rangle$, D accepts if M halts on $\langle M \rangle$ and rejects otherwise. Construct a new machine H: on input $\langle M \rangle$, run $D(\langle M \rangle)$; if D accepts, loop forever; if D rejects, halt and accept. Now consider H on input $\langle H \rangle$. If H halts on $\langle H \rangle$, then D accepts, so H loops. Contradiction. If H does not halt on $\langle H \rangle$, then D rejects, so H halts. Contradiction. Both cases fail, so D cannot exist. □

Not co-r.e.: If L_1 were both r.e. and co-r.e., it would be decidable (a language is decidable iff both it and its complement are r.e.). We showed L_1 is r.e. but not decidable, so L_1 can't be co-r.e. too. □

2. $L_2 = \{(\langle M \rangle, w) : M \textbf{ halts on input } w\}$. L_2 is the standard halting problem. It is r.e. but not decidable and not co-r.e.

R.e.: Construct a TM R that, on input $(\langle M \rangle, w)$, simulates M on w. If M halts, accept. If M loops, R loops. So R recognizes L_2.

Not decidable: Reduce from L_1. Given an instance $\langle M \rangle$ of L_1, construct the instance $(\langle M \rangle, \langle M \rangle)$ of L_2. Then M halts on itself iff $(\langle M \rangle, \langle M \rangle) \in L_2$. If L_2 were decidable, we could decide L_1 by querying the L_2 decider on $(\langle M \rangle, \langle M \rangle)$. Since L_1 is undecidable, L_2 must be undecidable too. □

Not co-r.e.: Same reasoning as L_1: L_2 is r.e. but not decidable, so it can't be co-r.e. either. □

3. $L_3 = \{(\langle M \rangle, w, k) : M \textbf{ halts on } w \textbf{ in at most } k \textbf{ steps}\}$. L_3 is decidable, so it's r.e. and co-r.e. too

Decidable: Construct a TM D: on input $(\langle M \rangle, w, k)$, simulate M on w for at most k steps. If M halts within k steps, accept. If M has not halted after k steps, reject. This machine always terminates because it runs for at most k simulation steps before deciding. It accepts when M halts on w within k steps, so D decides L_3. □

Unlike L_2, we're not asking "does M ever halt?" (which might run forever) but "does M halt within a bounded number of steps?" Bounded simulation always halts.

R.e. and co-r.e.: A decider is just a recognizer that also halts on non-members, so decidable implies r.e. Flip accept and reject to decide the complement, so decidable implies co-r.e. too. □

Exercise 2 (Big O properties). For non-negative functions $f, g, h \colon \mathbb{N} \to \mathbb{R}_{\geq 0}$, prove the following properties of Big O notation. Recall: $f \in \mathcal{O}(g)$ means there exist N and $M > 0$ such that $f(n) \leq M \cdot g(n)$ for all $n \geq N$.

Part 1: $f \in \mathcal{O}(f)$. Choose $M = 1$ and $N = 0$. Then $f(n) \leq 1 \cdot f(n)$ for all $n \geq 0$, which obviously holds. So $f \in \mathcal{O}(f)$. □

Part 2: $\mathcal{O}(c \cdot f) = \mathcal{O}(f)$ for any constant $c > 0$. I show both inclusions. ($\subseteq$): If $h \in \mathcal{O}(c \cdot f)$, then $h(n) \leq M \cdot c \cdot f(n) = (Mc) \cdot f(n)$ for $n \geq N$, so $h \in \mathcal{O}(f)$. ($\supseteq$): If $h \in \mathcal{O}(f)$, then $h(n) \leq M \cdot f(n) = (M/c) \cdot c \cdot f(n)$ for $n \geq N$, so $h \in \mathcal{O}(c \cdot f)$. □

Part 3: If $f \leq g$ for large enough inputs, then $\mathcal{O}(f) \subseteq \mathcal{O}(g)$. Let $h \in \mathcal{O}(f)$, so $h(n) \leq M \cdot f(n)$ for $n \geq N_1$. Since $f(n) \leq g(n)$ for $n \geq N_2$. Set $N = \max(N_1, N_2)$. For all $n \geq N$:

$$h(n) \leq M \cdot f(n) \leq M \cdot g(n),$$

so $h \in \mathcal{O}(g)$. □

Part 4: If $\mathcal{O}(f) \subseteq \mathcal{O}(g)$, then $\mathcal{O}(f + h) \subseteq \mathcal{O}(g + h)$. Since $f \in \mathcal{O}(f) \subseteq \mathcal{O}(g)$ (by Part 1), there exist M_1, N_1 with $f(n) \leq M_1 \cdot g(n)$ for $n \geq N_1$. Let $p \in \mathcal{O}(f + h)$, so $p(n) \leq M_2(f(n) + h(n))$ for $n \geq N_2$. Set $C = \max(M_1, 1)$ and $N = \max(N_1, N_2)$. For $n \geq N$:

$$p(n) \leq M_2(f(n) + h(n)) \leq M_2(M_1 g(n) + h(n)) \leq M_2 C \cdot (g(n) + h(n)),$$

so $p \in \mathcal{O}(g + h)$. □

Part 5: If $h(n) > 0$ for all n and $\mathcal{O}(f) \subseteq \mathcal{O}(g)$, then $\mathcal{O}(f \cdot h) \subseteq \mathcal{O}(g \cdot h)$. As in Part 4, $f \in \mathcal{O}(g)$ gives $f(n) \leq M_1 g(n)$ for $n \geq N_1$. Let $p \in \mathcal{O}(f \cdot h)$, so $p(n) \leq M_2 f(n)h(n)$ for $n \geq N_2$. Since $h(n) > 0$, multiplying $f(n) \leq M_1 g(n)$ by $h(n)$ keeps the inequality. For $n \geq \max(N_1, N_2)$:

$$p(n) \leq M_2 f(n)h(n) \leq M_2 M_1 g(n)h(n) = (M_1 M_2) \cdot g(n)h(n),$$

so $p \in \mathcal{O}(g \cdot h)$. □

Exercise 3 (Derived properties). Using the properties from Exercise 2, prove the following for $n, j, k \in \mathbb{N}$.

Part 1: If $j \leq k$, then $\mathcal{O}(n^j) \subseteq \mathcal{O}(n^k)$. For $n \geq 1$, $j \leq k$ implies $n^j \leq n^k$. By Exercise 2 Part 3, $\mathcal{O}(n^j) \subseteq \mathcal{O}(n^k)$. □

Part 2: If $j \leq k$, then $\mathcal{O}(n^j + n^k) \subseteq \mathcal{O}(n^k)$. For $n \geq 1$, $n^j \leq n^k$ gives $n^j + n^k \leq 2n^k$. By Exercise 2 Part 3, $\mathcal{O}(n^j + n^k) \subseteq \mathcal{O}(2n^k)$. By Exercise 2 Part 2, $\mathcal{O}(2n^k) = \mathcal{O}(n^k)$. Putting those together, $\mathcal{O}(n^j + n^k) \subseteq \mathcal{O}(n^k)$. □

Part 3: $\mathcal{O}\left(\sum_{i=0}^{k} a_i n^i\right) = \mathcal{O}(n^k)$, assuming $a_k > 0$. ($\subseteq$): For $n \geq 1$, each $n^i \leq n^k$ when $i \leq k$, so $\sum_{i=0}^{k} a_i n^i \leq \left(\sum_{i=0}^{k} |a_i|\right) n^k$. Let $C = \sum |a_i|$. By Parts 3 and 2, $\mathcal{O}(\text{poly}) \subseteq \mathcal{O}(Cn^k) = \mathcal{O}(n^k)$. ($\supseteq$): Since $a_k > 0$, for large enough n the leading term dominates and $\sum a_i n^i \geq \frac{a_k}{2} n^k$, so $n^k \leq \frac{2}{a_k} \sum a_i n^i$, giving $\mathcal{O}(n^k) \subseteq \mathcal{O}(\text{poly})$. □

Part 4: $\mathcal{O}(\log n) \subseteq \mathcal{O}(n)$. For $n \geq 1$, $\log n \leq n$ (since $x - \ln x$ is increasing for $x \geq 1$ and starts positive). By Exercise 2 Part 3, $\mathcal{O}(\log n) \subseteq \mathcal{O}(n)$. □

Part 5: $\mathcal{O}(n \log n) \subseteq \mathcal{O}(n^2)$. From Part 4, $\mathcal{O}(\log n) \subseteq \mathcal{O}(n)$. The function $h(n) = n$ satisfies $h(n) > 0$. By Exercise 2 Part 5, $\mathcal{O}(n \cdot \log n) \subseteq \mathcal{O}(n \cdot n) = \mathcal{O}(n^2)$. □

Exercise 4 (Comparing growth classes). Determine the relationship between each pair and prove the claim.

1. $\mathcal{O}(n)$ and $\mathcal{O}(\sqrt{n})$. *Claim:* $\mathcal{O}(\sqrt{n}) \subsetneq \mathcal{O}(n)$.

$\mathcal{O}(\sqrt{n}) \subseteq \mathcal{O}(n)$: For $n \geq 1$, $\sqrt{n} \leq n$, so by Exercise 2 Part 3 we are done.

$n \notin \mathcal{O}(\sqrt{n})$: Suppose $n \leq M\sqrt{n}$ for all $n \geq N$. Dividing by $\sqrt{n}$ gives $\sqrt{n} \leq M$ for all large n, but $\sqrt{n} \to \infty$ and M is a constant, contradiction. So $\mathcal{O}(n) \not\subseteq \mathcal{O}(\sqrt{n})$ and the inclusion is strict. □

2. $\mathcal{O}(n^2)$ and $\mathcal{O}(2^n)$. *Claim:* $\mathcal{O}(n^2) \subsetneq \mathcal{O}(2^n)$.

$\mathcal{O}(n^2) \subseteq \mathcal{O}(2^n)$: Polynomials always lose to exponentials eventually. $\lim_{n\to\infty} n^2/2^n = 0$, so $n^2 \leq 2^n$ for large enough n, and Exercise 2 Part 3 gives the inclusion.

$2^n \notin \mathcal{O}(n^2)$: Suppose $2^n \leq Mn^2$ for all $n \geq N$. Then $2^n/n^2 \leq M$, but $2^n/n^2 \to \infty$, contradiction. □

3. $\mathcal{O}(\log n)$ and $\mathcal{O}(\log^2 n)$. *Claim:* $\mathcal{O}(\log n) \subsetneq \mathcal{O}(\log^2 n)$.

$\mathcal{O}(\log n) \subseteq \mathcal{O}(\log^2 n)$: For $n \geq e$ (so that $\log n \geq 1$), we have $\log n \leq \log n \cdot \log n = \log^2 n$. By Exercise 2 Part 3, the inclusion holds.

$\log^2 n \notin \mathcal{O}(\log n)$: Suppose $\log^2 n \leq M \log n$ for all large n. Dividing by $\log n$ gives $\log n \leq M$, but $\log n \to \infty$, contradiction. □

4. $\mathcal{O}(2^n)$ and $\mathcal{O}(3^n)$. *Claim:* $\mathcal{O}(2^n) \subsetneq \mathcal{O}(3^n)$.

$\mathcal{O}(2^n) \subseteq \mathcal{O}(3^n)$: $2^n \leq 3^n$ for all $n \geq 0$ since $2 \leq 3$. By Exercise 2 Part 3, done.

$3^n \notin \mathcal{O}(2^n)$: Suppose $3^n \leq M \cdot 2^n$ for all $n \geq N$. Then $(3/2)^n \leq M$, but $(3/2)^n \to \infty$, contradiction. □

5. $\mathcal{O}(\log_2 n)$ and $\mathcal{O}(\log_3 n)$. *Claim:* $\mathcal{O}(\log_2 n) = \mathcal{O}(\log_3 n)$.

By the change-of-base formula, $\log_2 n = \frac{\log_3 n}{\log_3 2} = c \cdot \log_3 n$ where $c = 1/\log_3 2 > 0$ is a constant. By Exercise 2 Part 2, $\mathcal{O}(c \cdot \log_3 n) = \mathcal{O}(\log_3 n)$. So $\mathcal{O}(\log_2 n) = \mathcal{O}(\log_3 n)$. That's why we just write $\mathcal{O}(\log n)$ without a base. Different log bases only differ by a constant, and Big O absorbs constants. □

Exercise 5 (Sorting runtimes). Compare the runtimes of the following pairs of sorting algorithms using what we know about growth classes.

Bubble sort vs. insertion sort. Both have worst-case runtime $\mathcal{O}(n^2)$, so asymptotically they're the same. Bubble sort repeatedly swaps adjacent elements; insertion sort builds a sorted prefix one element at a time. Big O can't tell them apart. In practice, insertion sort tends to be faster because it makes fewer swaps and runs in $\mathcal{O}(n)$ on nearly-sorted inputs, while bubble sort still performs $\mathcal{O}(n^2)$ comparisons in most cases. Big O hides constant factors and best-case behavior though.

Insertion sort vs. merge sort. This is where Big O actually separates them. Insertion sort is $\Theta(n^2)$ worst case, merge sort is $\Theta(n \log n)$. Since $\mathcal{O}(n \log n) \subsetneq \mathcal{O}(n^2)$ (Exercise 3 Part 5), merge sort is in a strictly better class. For large inputs, merge sort always wins. For small inputs, insertion sort can be faster since there's no recursion or extra arrays. That's why Timsort uses insertion sort on small subarrays and merge sort for the rest.

Merge sort vs. quick sort. This one depends on which runtime measure we care about. Worst case, merge sort is $\Theta(n \log n)$ while quick sort is $\Theta(n^2)$ (when the pivot is always the smallest or largest element), so merge sort wins there. But in the average case (random pivot), quick sort is also $\Theta(n \log n)$, matching merge sort asymptotically. In practice quick sort is often faster due to better cache locality and smaller constant factors. Quick sort also uses $\mathcal{O}(\log n)$ auxiliary space (in-place, with tail-call optimization) compared to

merge sort's $\mathcal{O}(n)$. So Big O alone doesn't settle which algorithm is "better"; it depends on which metric matters and on the expected input distribution.

2.8.3 Exploration

Big O and decidability are both about drawing lines between things. Decidability asks "can this be solved at all?" and Big O asks "how fast can we solve it?" The L_3 exercise ties them together nicely: bounding the number of steps turns an undecidable problem into a decidable one. That's basically what Big O does too, it throws away the exact details and just keeps the shape of the growth.

The log base result from Exercise 4 is a good example of how aggressive that throwing-away is. $\log_2$ and $\log_3$ are different functions, but Big O says they're the same because the difference is just a constant. That's the whole point of the notation: ignore anything that doesn't change the growth rate.

2.8.4 Questions

Big O can't distinguish merge sort from quick sort in the average case. Is there a standard notation that also captures things like cache performance, or do people just benchmark?

2.9 Week 8

2.9.1 Notes

This section is optional.

2.9.2 Homework

Exercise 1 (CNF Conversion). Rewrite the following formulas in conjunctive normal form (CNF). Recall that a formula is in CNF if it is a conjunction of clauses, where each clause is a disjunction of literals.

Part 1: $\varphi_1 := \neg((a \wedge b) \vee (\neg c \wedge d))$**.** Apply De Morgan's law to the outer negation, $\neg(A \vee B) \equiv \neg A \wedge \neg B$:

$$\neg((a \wedge b) \vee (\neg c \wedge d)) = \neg(a \wedge b) \wedge \neg(\neg c \wedge d).$$

Apply De Morgan's again to each conjunct, $\neg(A \wedge B) \equiv \neg A \vee \neg B$:

$$= (\neg a \vee \neg b) \wedge (c \vee \neg d).$$

This is already in CNF: two clauses, each a disjunction of literals.

Part 2: $\varphi_2 := \neg((p \vee q) \to (r \wedge \neg s))$**.** First expand the implication using $A \to B := \neg A \vee B$:

$$(p \vee q) \to (r \wedge \neg s) = \neg(p \vee q) \vee (r \wedge \neg s).$$

So $\varphi_2 = \neg\big(\neg(p \vee q) \vee (r \wedge \neg s)\big)$. Apply De Morgan's:

$$= \neg\neg(p \vee q) \wedge \neg(r \wedge \neg s).$$

Double negation elimination on the first part, De Morgan's on the second:

$$= (p \vee q) \wedge (\neg r \vee s).$$

Already in CNF: two clauses.

Exercise 2 (Satisfiability). For each formula, determine whether it is satisfiable. If yes, give a satisfying assignment. If not, explain why no assignment exists.

Part 1: $\psi_1 := (a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b)$**.** Already in CNF with two variables and three clauses. Check all four assignments:

a	b	$(a \vee \neg b)$	$(\neg a \vee b)$	$(\neg a \vee \neg b)$	ψ_1
0	0	1	1	1	**1**
0	1	0	1	1	0
1	0	1	0	1	0
1	1	1	1	0	0

Satisfiable. The unique satisfying assignment is $\mathfrak{J}(a) = 0$, $\mathfrak{J}(b) = 0$.

Part 2: $\psi_2 := (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg(\neg p \vee r)$**.** First simplify the third conjunct using De Morgan's law:

$$\neg(\neg p \vee r) = p \wedge \neg r.$$

So $\psi_2 = (\neg p \vee q) \wedge (\neg q \vee r) \wedge p \wedge \neg r$. Now propagate the unit clauses:

- From p: we must have $p = 1$.
- From $\neg r$: we must have $r = 0$.
- Substituting $r = 0$ into $(\neg q \vee r)$ gives $\neg q$, so $q = 0$.
- Substituting $p = 1$, $q = 0$ into $(\neg p \vee q)$ gives $(0 \vee 0) = 0$. Contradiction.

Not satisfiable. The unit clauses force $p = 1$, $r = 0$, $q = 0$, which falsifies $(\neg p \vee q)$. No assignment can make all four conjuncts true simultaneously.

Part 3: $\psi_3 := (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$**.** Use resolution. Resolving clauses 1 and 2 on x:

$$(x \vee y) \wedge (\neg x \vee y) \implies y.$$

Resolving clauses 3 and 4 on x:

$$(x \vee \neg y) \wedge (\neg x \vee \neg y) \implies \neg y.$$

The first pair forces $y = 1$ and the second forces $y = 0$. Contradiction.

Not satisfiable. You can also verify by checking all four assignments of (x, y); each one falsifies at least one clause.

2.9.3 Exploration

Decidability asks whether a problem can be solved at all. Complexity asks how long the solution takes. P and NP both sit inside the decidable languages, but the distinction between them is about polynomial time versus brute-force search. A problem being decidable doesn't mean much if the best algorithm takes 2^n steps.

The homework exercises were small enough to solve by hand. Two or three variables means four or eight rows in a truth table. But SAT in general has n variables and 2^n possible assignments. Last week we showed that exponentials dominate polynomials ($\mathcal{O}(n^k) \subsetneq \mathcal{O}(2^n)$), and that gap is exactly why SAT is considered intractable. Real SAT instances have thousands of variables, and brute force stops being feasible fast.

Converting a formula to CNF is mechanical: just apply De Morgan's laws and double negation elimination. That matters because SAT solvers assume CNF input. The conversion itself is cheap, so the bottleneck is always the solving, not the preprocessing. The Cook-Levin theorem says that bottleneck is as hard as any problem in NP.

2.9.4 Questions

SAT is NP-complete, so no known algorithm solves it in polynomial time. Why do SAT solvers work so well in practice if the worst case is exponential?

3 Synthesis

(approx 1 page, plus references)Section 2 gives you an opportunity to practice "skill drill" and to explore the material in more depth. The purpose of this section is to synthesize the knowledge you gained. Since Programming Languages is a wide field, it may be appropriate to focus on a particular topic of your choice.

We suggest the following timeline. Week 5-8: Decide on a topic and discuss it with your instructor in the office hours. Write a summary email to your instructor after office hours, including the feedback you got with further reflection and planning. Week 9-12: Write a draft of your synthesis and discuss it with your instructor during office hours. Again, write a summary email to your instructor after office hours, including the feedback you got with further reflection and planning. The final version of the Synthesis is due with the rest of the final report.

Here are some example titles you can think of for this section:

- **Standard Academic Titles**
  - "Synthesis and Reflection"
  - "Integration of Concepts"
  - "Synthesis of Learning"
  - "Critical Synthesis"
- **Algorithm Analysis Focus**
  - "The Core of the Code: A Personal Synthesis"
  - "Connecting the Dots: From Theory to Practice"
  - "The Big Picture: Algorithm Analysis in Context"
  - "Principles of Problem-Solving"
- **Concept-Focused Titles**
  - "Beyond the Code: Understanding the Essence of Algorithms"
  - "How long is too long? A Case Study in Complexity Theory"
  - "Principles of Computation"
  - "Abstract Machines for Concrete Problems"
  - "Is there an Algorithm for this? On (Un)Decidability"
- **Process-Focused Titles**
  - "My Journey Through Algorithm Analysis"
  - "From Implementation to Understanding"
  - "Building Mental Models of Practical Problems"
  - "Discovering the Foundations behind Algorithms"

4 Evidence of Participation

5 Conclusion

(approx 400 words) A critical reflection on the content of the course. Step back from the technical details. How does the course fit into the wider world of software engineering? What did you find most interesting or useful? What improvements would you suggest?

References

[BLA] Author, Title, Publisher, Year.